

```

#include <cstdlib>
#include <iostream>
#include <stdexcept>
#include <ctime>
#include <unordered_map>
#include <queue>
#include <vector>
#include <limits>

template <class T>
class Node
{
public:
    Node()
    {
        data = new T;
    }
    Node(T d)
    {
        data = new T;
        (*data) = d;
    }
    Node &operator=(T d)
    {
        (*data) = d;
        return *this;
    }
    friend std::ostream &operator<<(std::ostream &out, Node n)
    {
        out<<*(n.data);
        return out;
    }
    friend std::ostream &operator<<(std::ostream &out, Node *n)
    {
        out<<*(n->data);
        return out;
    }
    void setData(T d)
    {
        *data = d;
    }
    T &getData() const
    {
        return *data;
    }
protected:
    T *data;
};

template<class T>
class ListNode : public Node<T>
{
public:
    ListNode() : Node<T>()
    {
        prev = NULL;
        next = NULL;
    }
    ListNode(T d) : Node<T>(d)
    {

```

```

        prev = NULL;
        next = NULL;
    }
    ListNode(ListNode *p, ListNode *n) : Node<T>()
    {
        prev = p;
        next = n;
    }
    ListNode(T d, ListNode *p, ListNode *n) : Node<T>(d)
    {
        prev = p;
        next = n;
    }
    ListNode *getNext() const
    {
        return next;
    }
    ListNode *getPrev() const
    {
        return prev;
    }
    void setNext(ListNode *n)
    {
        next = n;
    }
    void setPrev(ListNode *p)
    {
        prev = p;
    }
    ListNode &operator=(T d)
    {
        this->setData(d);
        return *this;
    }
private:
    ListNode *prev, *next;
};

```

```

template<class T>
class LinkList
{
public:
    LinkList()
    {
        head = NULL;
        tail = NULL;
    }
    void addFromHead(T d)
    {
        ListNode<T> *node = new ListNode<T>(d);
        if(head != NULL)
        {
            head->setPrev(node);
        }
        node->setNext(head);
        head = node;
        if(tail == NULL)
            tail = node;
    }
    void addFromTail(T d)

```

```

{
    ListNode<T> *node = new ListNode<T>(d);
    if(tail != NULL)
    {
        tail->setNext(node);
    }
    node->setPrev(tail);
    tail = node;
    if(head == NULL)
        head = node;
}

void addAfter(ListNode<T> *at, T d)
{
    if(!exist(at))
        return;
    ListNode<T> *node = new ListNode<T>(d);
    if(at->getNext() != NULL)
        at->getNext()->setPrev(node);
    node->setNext(at->getNext());
    at->setNext(node);
    node->setPrev(at);
    if(node->getNext() == NULL)
        tail = node;
}

ListNode<T> *removeFromHead()
{
    ListNode<T> *n = head;
    if(head != NULL)
    {
        head = head->getNext();
        if(head != NULL)
            head->setPrev(NULL);
        else
            tail = NULL;
        n->setNext(NULL);
    }
    return n;
}

ListNode<T> *removeFromTail()
{
    ListNode<T> *n = tail;
    if(tail != NULL)
    {
        tail = tail->getPrev();
        if(tail != NULL)
            tail->setNext(NULL);
        else
            head = NULL;
        n->setPrev(NULL);
    }
    return n;
}

ListNode<T> *remove(ListNode<T> *n)
{
    if(!exist(n))
        return NULL;
    if(n == head)
        return removeFromHead();
    if(n == tail)
        return removeFromTail();
}

```

```

        n->getPrev()->setNext(n->getNext());
        n->getNext()->setPrev(n->getPrev());
        n->setNext(NULL);
        n->setPrev(NULL);
        return n;
    }
    ListNode<T> *exist(T d)
    {
        ListNode<T> *j = head;
        while(j != NULL)
        {
            if(j->getData() == d)
                return j;
            j = j->getNext();
        }
        return NULL;
    }
    bool exist(ListNode<T> *n)
    {
        ListNode<T> *j = head;
        while(j != NULL)
        {
            if(j == n)
                return true;
            j = j->getNext();
        }
        return false;
    }
    ListNode<T> &operator[](int i)
    {
        ListNode<T> *j = head;
        int k;
        for(k = 0; k < i && j != NULL; k++)
            j = j->getNext();
        if(j == NULL)
            throw std::invalid_argument("index does not exist.");
        return *j;
    }
    void print() const
    {
        ListNode<T> *j;
        j = head;
        while(j != NULL)
        {
            std::cout<<(*j)<<" ";
            j = j->getNext();
        }
        std::cout<<std::endl;
    }
protected:
    ListNode<T> *head, *tail;
};

template<class V, class E>
class WeightedGraphEdge;

template<class V, class E>
class WeightedGraphVertex : public Node<V>
{
public:

```

```

WeightedGraphVertex() :Node<V>()
{
    list = new LinkedList<E>();
}
WeightedGraphVertex(V d) :Node<V>(d)
{
    list = new LinkedList<WeightedGraphEdge<V, E>*>();
}
void addEdge(WeightedGraphEdge<V, E> *edge)
{
    list->addFromTail(edge);
}
void remove(WeightedGraphEdge<V, E> *node)
{
}
ListNode<WeightedGraphEdge<V, E>*> *operator[](int n)
{
    try
    {
        return (&(*list)[n]);
    }
    catch(std::invalid_argument e)
    {
        return NULL;
    }
}
private:
    LinkedList<WeightedGraphEdge<V, E>*> *list;
};

template<class V, class E>
class WeightedGraphEdge : public Node<E>
{
public:
    WeightedGraphEdge() : Node<E>()
    {
        end[0] = NULL;
        end[1] = NULL;
    }
    WeightedGraphEdge(E d) : Node<E>(d)
    {
        end[0] = NULL;
        end[1] = NULL;
    }
    WeightedGraphEdge(E d, WeightedGraphVertex<V, E> *v1, WeightedGraphVertex<V, E>
*v2) : Node<E>(d)
    {
        end[0] = v1;
        end[1] = v2;
    }
    WeightedGraphVertex<V, E> *getAnotherEnd(WeightedGraphVertex<V, E> *v)
    {
        if(v != end[0])
            return end[0];
        return end[1];
    }
private:
    WeightedGraphVertex<V, E> *end[2];
};

```

```

template<class V, class E>
class WeightedGraph
{
public:
    WeightedGraph()
    {
        vertex = new LinkedList<WeightedGraphVertex<V, E> *>();
        edge = new LinkedList<WeightedGraphEdge<V, E> *>();
    }
    WeightedGraphVertex<V, E> *operator[](int n)
    {
        try
        {
            return (*vertex)[n].getData();
        }
        catch(std::invalid_argument e)
        {
            return NULL;
        }
    }
    void addLink(WeightedGraphVertex<V, E> *v1, WeightedGraphVertex<V, E> *v2, E w)
    {
        WeightedGraphEdge<V, E> *edge = new WeightedGraphEdge<V, E>(w, v1, v2);
        v1->addEdge(edge);
        if(v1 != v2)
            v2->addEdge(edge);
    }
    WeightedGraphVertex<V, E> *addVertex(V d)
    {
        WeightedGraphVertex<V, E> *v = new WeightedGraphVertex<V, E>(d);
        vertex->addFromTail(v);
        vertexCount ++;
        return v;
    }
    void adjList()
    {
        ListNode<WeightedGraphVertex<V, E> *> *cur = &(*vertex)[0];
        while(cur != NULL)
        {
            WeightedGraphVertex<V, E> *temp = cur->getData();
            std::cout<<temp<<": ";
            ListNode<WeightedGraphEdge<V, E> *> *e = (*temp)[0];
            while(e != NULL)
            {
                std::cout<<e->getData()->getAnotherEnd(temp)<<"(" <<e-
>getData()<<") ";
                e = e->getNext();
            }
            std::cout<<std::endl;
            cur = cur->getNext();
        }
    }
    /*
    return null if n is not a vertex in this graph
    return the shorest path tree with v as root
    */
    WeightedGraph *shortestPathTree(WeightedGraphVertex<V, E> *v)
    {
        if (!v) return nullptr;
    }

```

```

    WeightedGraph<V, E> *tree = new WeightedGraph<V, E>();
    std::unordered_map<WeightedGraphVertex<V, E> *, E> distances;
    std::unordered_map<WeightedGraphVertex<V, E> *, WeightedGraphEdge<V, E> *>
shortestEdges;
    std::unordered_map<WeightedGraphVertex<V, E> *, bool> visited;
    auto cmp = [&distances](WeightedGraphVertex<V, E> *a, WeightedGraphVertex<V, E> *b) {
        return distances[a] > distances[b];
    };
    std::priority_queue<WeightedGraphVertex<V, E> *, std::vector<WeightedGraphVertex<V, E>
*>, decltype(cmp)> pq(cmp);

    ListNode<WeightedGraphVertex<V, E> *> *cur = &(*vertex)[0];
    while (cur != nullptr)
    {
        distances[cur->getData()] = std::numeric_limits<E>::max();
        visited[cur->getData()] = false;
        cur = cur->getNext();
    }

    distances[v] = 0;
    pq.push(v);

    while (!pq.empty())
    {
        WeightedGraphVertex<V, E> *current = pq.top();
        pq.pop();
        if (visited[current]) continue;
        visited[current] = true;

        tree->addVertex(current->getData());

        if (shortestEdges[current])
        {
            tree->addLink(shortestEdges[current]->getAnotherEnd(current), current,
shortestEdges[current]->getData());
        }

        ListNode<WeightedGraphEdge<V, E> *> *edge = (*current)[0];
        while (edge != nullptr)
        {
            WeightedGraphEdge<V, E> *e = edge->getData();
            WeightedGraphVertex<V, E> *neighbor = e->getAnotherEnd(current);

            if (!visited[neighbor] && distances[current] + e->getData() < distances[neighbor])
            {
                distances[neighbor] = distances[current] + e->getData();
                shortestEdges[neighbor] = e;
                pq.push(neighbor);
            }

            edge = edge->getNext();
        }
    }

    return tree;
}

private:
    LinkedList<WeightedGraphVertex<V, E> *> *vertex;
    LinkedList<WeightedGraphEdge<V, E> *> *edge;

```

```

        int vertexCount, edgeCount;
    };

int main()
{
    WeightedGraph<char, int> *g = new WeightedGraph<char, int>();
    WeightedGraph<char, int> *tree;
    int j, k, i, l;
    srand(time(NULL));
    for(j = 0; j < 5; j++)
    {
        g->addVertex(j + 'a');
    }
    for(j = 0; j < 5; j++)
    {
        k = rand() % 5;
        i = rand() % 5;
        l = rand() % 100;
        g->addLink((*g)[k], (*g)[i], l);
    }
    g->adjList();
    tree = g->shortestPathTree((*g)[0]);
    tree->adjList();
    return 0;
}

```